Supplementary Material for Beyond Active Learning: Implicit Recommendation and Diagnostic Benchmarks for Exploratory Data Analysis

David Adams Nir Lipovetzky Renata Borovica-Gajic

Contents

A Additional Experimental Details **2**
A.1 Benchmarks and Archived Runs . . . 2
A.2 Prediction Gaps and Evaluation Counts . . . 2
A.3 Gap-1 Metric Tables . . . 3
A.4 Gap-Wise F1 Tables . . . 3
A.5 Additional Figures . . . 4

B Reference for `tools/generate_benchmark.py` **6**
B.1 Purpose and Invocation . . . 6
B.2 Synthetic Schema . . . 6
B.3 Embedded Association Rules . . . 6
B.4 Generation Pipeline . . . 6
B.5 Output Layout . . . 7
B.6 Reference Configuration . . . 7
B.7 Why the Generator Is Diagnostic . . . 7

C Theoretical Analysis: Workload–Recommender Alignment **10**
C.1 Formal Session Model . . . 10
C.2 Evaluation: nDCG as the Quality Criterion . . . 10
C.3 Recommender Scoring Functions . . . 11
C.4 Four Workload Parameters . . . 12
C.5 Failure Conditions for Baseline Recommenders . . . 12
C.6 MDI's Temporal Accumulation Advantage . . . 13
C.7 Benchmark Construction: A Prescriptive Recipe . . . 13

A Additional Experimental Details

The experimental record includes the benchmark configurations, the number of evaluated query pairs at each prediction gap, the archived gap-wise F1 tables, and the supporting figures.

A.1 Benchmarks and Archived Runs

Table 1 summarizes the three workloads used in the evaluation. The archived result directories and analysis CSVs under `previous-results/paper/` provide the stable source for the tables that follow.

Table 1: Workloads used in the evaluation.

Workload	Primary Source	Sessions	Queries	Pattern
SIMBA drill-down	`data/datasets/simba_drilldown`	1	20	Incremental drill-down
Adversarial benchmark	`data/datasets/benchmark_mdi`	3	about 28 per session	Rare-rule signal
SkyServer SQL logs	Archived paper sample	10 sampled	981 before gap filtering	Point lookups, abrupt shifts

The checked-in configuration files are:

- SIMBA: `experiments/configs/simba_drilldown.yml`
- Adversarial benchmark: `experiments/configs/benchmark_mdi_vs_baselines.yml`
- SkyServer SQL logs: `experiments/configs/sdss_vs_baselines.yml`

For the SkyServer workload, the archived analysis under `previous-results/paper/sdss-logs/` contains gap summaries for $g \in \{1, 2, 3, 5, 10\}$. The current example configuration file lists a narrower gap range, so the tables use the archived analysis rather than the smaller checked-in example.

A.2 Prediction Gaps and Evaluation Counts

Table 2 shows how many query pairs remain for each workload after applying the prediction-gap filter. These counts come directly from the archived `all_sessions_metrics.csv` files and explain why the SIMBA curve becomes unstable at large gaps: only five query pairs remain at $g = 10$.

Table 2: Number of evaluated query pairs per workload and prediction gap.

Workload	$g = 1$	$g = 2$	$g = 3$	$g = 5$	$g = 10$
SIMBA drill-down	14	13	12	10	5
Adversarial benchmark	87	84	81	75	60
SkyServer SQL logs	666	527	520	506	471

All experiments use the same memory-based evaluation protocol. At step q and gap g, a recommender ranks tuples in the current result set R_q, and tuples that reappear in R_{q+g} are treated as relevant. The recommendation budget is set to the size of the future result so that all recommenders are compared under the same output cardinality.

A.3 Gap-1 Metric Tables

Table 3 reports the full precision, recall, F1, and overlap values for the gap-1 results across all three workloads.

Table 3: Gap-1 metrics across the three workloads.

Workload	**Recommender**	**Precision**	**Recall**	**F1**	**Overlap**
SIMBA drill-down	MDI (Ours)	0.292	0.283	0.287	0.292
SIMBA drill-down	Similarity	0.340	0.330	0.335	0.340
SIMBA drill-down	Frequency	0.308	0.299	0.303	0.308
SIMBA drill-down	Random	0.302	0.292	0.297	0.302
SIMBA drill-down	Clustering	0.292	0.283	0.287	0.000
SIMBA drill-down	Sampling	0.271	0.261	0.266	0.271
Adversarial benchmark	MDI (Ours)	0.292	0.249	0.260	0.295
Adversarial benchmark	Clustering	0.248	0.183	0.199	0.248
Adversarial benchmark	Random	0.239	0.176	0.191	0.243
Adversarial benchmark	Sampling	0.239	0.176	0.191	0.243
Adversarial benchmark	Similarity	0.208	0.145	0.160	0.212
Adversarial benchmark	Frequency	0.198	0.135	0.150	0.202
SkyServer SQL logs	Clustering	0.037	0.033	0.033	0.037
SkyServer SQL logs	Frequency	0.037	0.033	0.033	0.037
SkyServer SQL logs	Random	0.037	0.033	0.033	0.037
SkyServer SQL logs	Sampling	0.037	0.033	0.033	0.037
SkyServer SQL logs	Similarity	0.019	0.018	0.018	0.019
SkyServer SQL logs	MDI (Ours)	0.014	0.012	0.013	0.014

A.4 Gap-Wise F1 Tables

Gap-1 tables alone hide how performance changes as the horizon grows. The next three tables report archived mean F1 values at all evaluated gaps.

Table 4: Mean F1 by prediction gap on the adversarial benchmark. MDI remains the top method at every archived gap. Relative improvement over random is 36.3% at $g = 1$, 43.7% at $g = 2$, 0.3% at $g = 3$, 18.5% at $g = 5$, and 14.3% at $g = 10$.

Recommender	$g = 1$	$g = 2$	$g = 3$	$g = 5$	$g = 10$
MDI (Ours)	0.260	0.202	0.153	0.153	0.150
Clustering	0.199	0.143	0.151	0.132	0.136
Random	0.191	0.140	0.152	0.129	0.131
Sampling	0.191	0.134	0.150	0.125	0.132
Similarity	0.160	0.110	0.144	0.115	0.126
Frequency	0.150	0.106	0.144	0.114	0.122

Interpretation. The adversarial benchmark is the only workload that cleanly separates the recommenders across multiple horizons. MDI is best at every archived gap, and its largest margin over random occurs at $g = 2$, matching the intuition that the synthetic sessions retain short-range

Table 5: Mean F1 by prediction gap on the SIMBA drill-down workload. The workload is discriminative only at small gaps; by $g = 10$, no evaluated pair retains overlap.

Recommender	$g = 1$	$g = 2$	$g = 3$	$g = 5$	$g = 10$
MDI (Ours)	0.287	0.137	0.046	0.009	0.000
Clustering	0.287	0.144	0.103	0.042	0.000
Random	0.297	0.137	0.058	0.009	0.000
Sampling	0.266	0.153	0.070	0.009	0.000
Similarity	0.335	0.189	0.030	0.009	0.000
Frequency	0.303	0.200	0.080	0.009	0.000

Table 6: Mean F1 by prediction gap on the archived SkyServer SQL-log analysis. All methods remain near zero because tuple persistence is weak and many transitions behave like entity jumps rather than refinements.

Recommender	$g = 1$	$g = 2$	$g = 3$	$g = 5$	$g = 10$
MDI (Ours)	0.013	0.043	0.023	0.012	0.014
Clustering	0.033	0.058	0.048	0.033	0.026
Random	0.033	0.059	0.048	0.033	0.026
Sampling	0.033	0.059	0.048	0.033	0.026
Similarity	0.018	0.048	0.038	0.023	0.022
Frequency	0.033	0.061	0.048	0.033	0.026

rule persistence before all methods are penalized by longer temporal distance. SIMBA behaves differently: the gap-1 results look competitive for almost every method because progressive drill-down preserves overlap, but the task quickly collapses as the gap increases. The SkyServer workload is the opposite failure mode. It remains low-signal at every gap because many query transitions are not refinements of the current result.

A.5 Additional Figures

Figures 1 and 2 show the gap analysis and the decay-sensitivity curve for the adversarial benchmark.

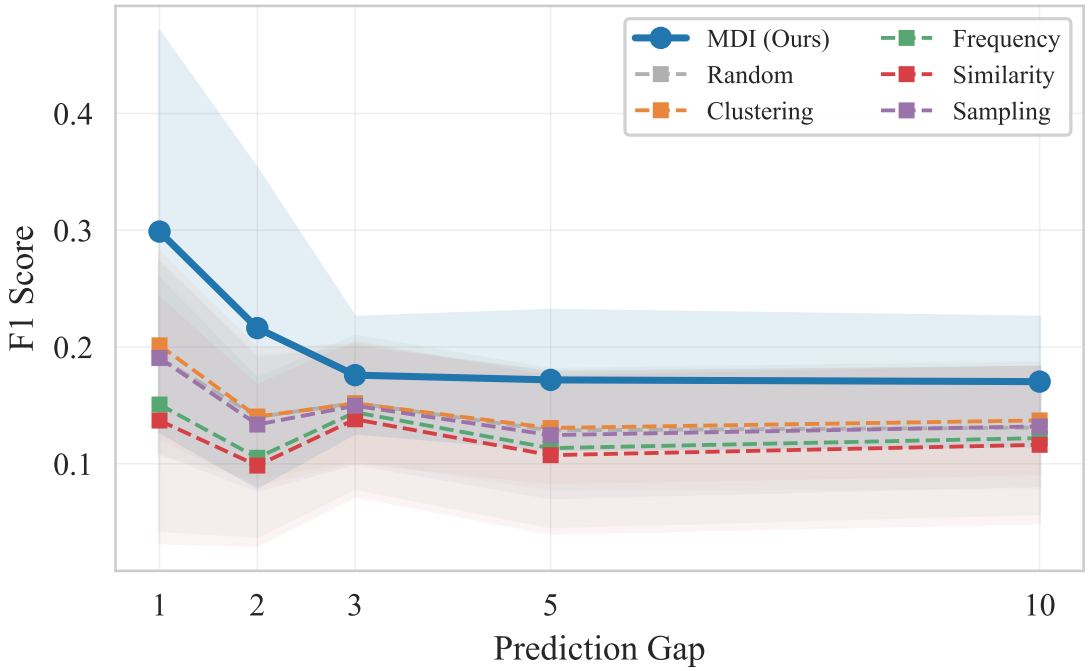

Figure 1: F1 across prediction gaps on the adversarial benchmark. MDI maintains the largest advantage at gaps 1 and 2, then remains modestly ahead as overlap decays.

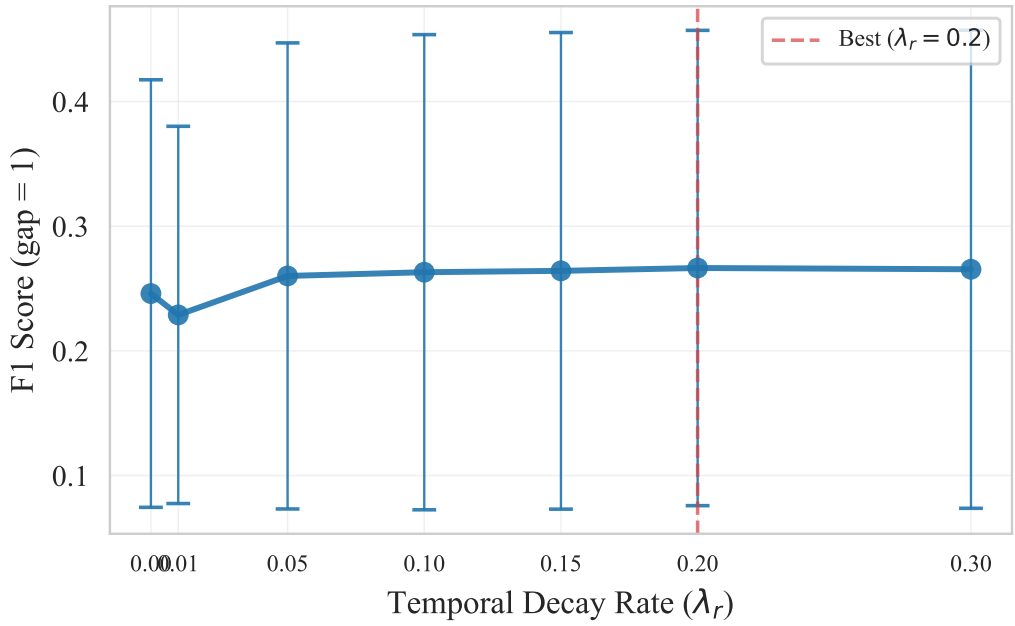

Figure 2: Sensitivity of MDI to the temporal decay rate λ_r on the adversarial benchmark at $g = 1$. The best mean F1 occurs around $\lambda_r = 0.20$, but the curve stays relatively flat across $\lambda_r \in [0.05, 0.30]$.

B Reference for `tools/generate_benchmark.py`

The synthetic benchmark generator produces the controlled adversarial workload. Its structure is diagnostic: it rewards recommenders that accumulate association evidence over time and systematically disadvantages baselines that rely on centroid similarity, attribute frequency, or uniform random selection.

B.1 Purpose and Invocation

The script supports two primary modes: dataset generation and self-verification. The generation mode writes a synthetic dataset in the format expected by the project's `DataLoader`, along with a matching experiment configuration. The verification mode constructs a smaller dataset in a temporary directory, instantiates all six recommenders, and reports their precision on several gaps as a quick regression test.

Listing 1: Typical invocations of the benchmark generator.

```
python tools/generate_benchmark.py \
  -o data/datasets/benchmark_mdi \
  -n 3 \
  -q 30 \
  --seed 42

python tools/generate_benchmark.py --verify --seed 42
```

B.2 Synthetic Schema

The benchmark uses a nine-attribute categorical schema. Table 8 lists the full domains. Noise rows concentrate on a small subset of common values in each column, while rule-conforming rows use combinations that are rare in the noise distribution.

B.3 Embedded Association Rules

The generator embeds five explicit association rules. Each rule is implemented as a pair of dictionaries (X, Y) where X is the antecedent and Y is the consequent. Rule rows are generated by sampling a uniform background tuple, forcing the antecedent values, and then applying the consequent with confidence 0.85.

B.4 Generation Pipeline

The script is organized around three helper classes and two top-level helper functions.

1. **`BenchmarkSchemaGenerator`** builds the fact table. It allocates about 12% of rows to each embedded rule, generates the remaining rows as biased noise, and explicitly perturbs any accidental noise row that happens to satisfy a rule antecedent.

2. **`QuerySequenceGenerator`** constructs each synthetic session. Large queries contain about 140–180 tuples and are dominated by noise, but they also include a persistent core of rule tuples. Small queries contain about 38–50 tuples and overlap with the previous large query mostly through rule-conforming tuples.

3. **`DatasetWriter`** serializes the sessions into the project format: one `metadata.csv`, one pickled session dataframe per session, and one pickled result dataframe per query under `query_results/`.

4. **`generate_experiment_config`** writes a ready-to-run YAML file for the adversarial benchmark. The generated configuration evaluates the six recommenders at gaps 1, 2, 3, 5, and 10.

5. **`run_verify`** is a self-test harness. It regenerates a small benchmark, validates the file layout, instantiates the recommender classes, and reports per-gap precision so that the generator can be checked without running the full experiment pipeline.

The large-query and small-query pattern is the crucial design choice. The script marks positions divisible by 3 or 5 as small queries, so gaps 1, 2, 3, 5, and 10 include many large-to-small transitions. Those are the pairs where the benchmark is deliberately discriminative: the future result is small enough that rule overlap matters, but the current result is large enough that random or centroid-based methods can easily miss the signal.

B.5 Output Layout

The generated SQL strings are synthetic placeholders of the form `SELECT * FROM orders WHERE region = ... AND category = ...`. They do not attempt to regenerate a full executable SQL workload; their role is simply to provide a stable textual field for downstream tooling that expects query text.

B.6 Reference Configuration

The generated YAML sets the recommender list to `multidimensional_interestingness`, `random`, `clustering`, `frequency`, `similarity`, and `sampling`. The default MDI settings in the generator match the archived adversarial benchmark: $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$, rule decay rate $\lambda_r = 0.05$, minimum support 0.08, and evaluation gaps $\{1, 2, 3, 5, 10\}$.

B.7 Why the Generator Is Diagnostic

The benchmark is intentionally asymmetric with respect to recommender families. It does not claim to represent all real EDA workloads. It isolates one capability: identifying tuples that satisfy rare but persistent associations in the presence of common-value noise. The script enforces four workload conditions:

- low effective signal fraction inside large queries,
- strong frequency skew caused by the 0.80 common-value bias,
- low noise variance because most noise rows collapse onto the same dominant values, and
- enough session depth for association evidence to accumulate across repeated large-query exposures.

Accordingly, `generate_benchmark.py` functions as a workload-construction tool rather than a generic data synthesizer.

Table 7: Command-line interface of `generate_benchmark.py`.

Argument	Default	Meaning
`-o`, `-output`	`data/datasets/benchmark_mdi`	Output directory for the generated benchmark dataset
`-n`, `-num-sessions`	3	Number of synthetic sessions to create
`-q`, `-queries-per-session`	30	Number of result sets per session
`-total-rows`	5000	Number of rows in the synthetic fact table
`-seed`	42	Random seed used for all generators
`-verify`	false	Run the built-in self-test instead of writing a persistent dataset
`-config-output`	`experiments/configs/benchmark_mdi_vs_baselines.yml`	Location of the generated experiment YAML
`-v`, `-verbose`	false	Enable debug logging

Table 8: Schema used by the synthetic benchmark generator.

Attribute	Domain
`region`	ASIA, EUROPE, AMERICA, AFRICA, MIDDLE_EAST
`category`	TECHNOLOGY, AUTOMOTIVE, HEALTHCARE, FINANCE, RETAIL, ENERGY, AGRICULTURE, LOGISTICS
`supplier_type`	DOMESTIC, INTERNATIONAL, PREMIUM, BUDGET
`price_range`	LOW, MEDIUM, HIGH, PREMIUM
`quantity_bin`	SMALL, MEDIUM, LARGE, BULK
`priority`	LOW, STANDARD, URGENT
`ship_mode`	AIR, SEA, RAIL, TRUCK
`year`	2022, 2023, 2024
`quarter`	Q1, Q2, Q3, Q4

Table 9: Noise-biased common values used by the generator. With probability 0.80, a noise row draws from these dominant values instead of the full domain.

Attribute	**Dominant Noise Values**
`region`	AMERICA, EUROPE
`category`	AUTOMOTIVE, FINANCE
`supplier_type`	DOMESTIC, BUDGET
`price_range`	MEDIUM, LOW
`quantity_bin`	MEDIUM, SMALL
`priority`	STANDARD, LOW
`ship_mode`	TRUCK, RAIL
`year`	2023, 2024
`quarter`	Q2, Q3

Table 10: Embedded association rules in `generate_benchmark.py`.

Rule	**Antecedent**	**Consequent**
1	`region=ASIA`, `category=TECHNOLOGY`	`price_range=HIGH`, `priority=URGENT`
2	`region=EUROPE`, `category=HEALTHCARE`	`price_range=PREMIUM`, `ship_mode=AIR`
3	`region=MIDDLE_EAST`, `category=RETAIL`	`price_range=PREMIUM`, `quantity_bin=LARGE`
4	`supplier_type=INTERNATIONAL`, `category=ENERGY`	`quantity_bin=BULK`, `ship_mode=SEA`
5	`region=AFRICA`, `category=AGRICULTURE`	`price_range=HIGH`, `priority=URGENT`

Table 11: Files produced by the benchmark generator.

Path	**Content**
`metadata.csv`	Dataset index with one row per session and columns `session_id` and `filepath`
`query_prediction_session_<session>.pkl`	Pickled session dataframe with `session_id`, `query_position`, `results_filepath`, and `current_query`
`query_results/results_session_<session>_query_<q>.pkl`	Pickled dataframe containing the tuples for query position q
`experiments/configs/benchmark_mdi_vs_baselines.yml`	Generated experiment configuration that points at the new dataset

C Theoretical Analysis: Workload–Recommender Alignment

A formal workload model makes the benchmark conditions explicit. The analysis defines what each recommender optimizes and derives workload regimes under which each method succeeds or fails. The same derivation explains the benchmark design and yields a recipe for controlled evaluations targeted at specific recommender capabilities.

C.1 Formal Session Model

Schema and tuples. Let $\mathcal{A} = \{A_1, \ldots, A_d\}$ be a fixed relational schema. Each attribute A_i has a finite domain $\mathrm{dom}(A_i)$. A *tuple* $t \in \mathcal{T}$ is a complete assignment $t : \mathcal{A} \to \bigcup_i \mathrm{dom}(A_i)$ such that $t(A_i) \in \mathrm{dom}(A_i)$. The *fact table* $\mathcal{T}$ is a finite multiset of such tuples.

Association rules. An *association rule* $r = (X \Rightarrow Y)$ is a pair of attribute-value sets $X, Y \subset \{A_i = v \mid A_i \in \mathcal{A},\, v \in \mathrm{dom}(A_i)\}$ with $X \cap Y = \emptyset$. A tuple t *satisfies the antecedent* X if $t(A_i) = v$ for all $(A_i = v) \in X$, and similarly for Y. Standard interestingness measures are:

$$\mathrm{supp}(r) = \frac{|\{t \in \mathcal{T} \mid t \models X \cup Y\}|}{|\mathcal{T}|}, \tag{1}$$

$$\mathrm{conf}(r) = \frac{\mathrm{supp}(r)}{\mathrm{supp}(X)}, \tag{2}$$

$$\mathrm{lift}(r) = \frac{\mathrm{conf}(r)}{\mathrm{supp}(Y)}, \tag{3}$$

where $\mathrm{supp}(X)$ (resp. $\mathrm{supp}(Y)$) denotes the fraction of tuples satisfying X (resp. Y) alone. The *J-measure* [1] is a mutual-information proxy:

$$J(r) = \mathrm{supp}(r) \log \frac{\mathrm{conf}(r)}{\mathrm{supp}(Y)} + (1 - \mathrm{supp}(r)) \log \frac{1 - \mathrm{conf}(r)}{1 - \mathrm{supp}(Y)}. \tag{4}$$

EDA session and prediction task. An *EDA session* of length Q is an ordered sequence of query results $\mathcal{S} = (R_0, R_1, \ldots, R_{Q-1})$ where each $R_q \subseteq \mathcal{T}$ is a finite multiset. The *history* at step q is $\mathcal{H}_q = (R_0, \ldots, R_{q-1})$.

At each step q with *prediction gap* $g \geq 1$, a recommender is given $(R_q, \mathcal{H}_q)$ and must rank the tuples of R_q. The ground-truth relevance label for a tuple $t \in R_q$ is

$$\mathrm{rel}(t, q, g) = \mathbf{1}\big[t \in R_{q+g}\big]. \tag{5}$$

That is, a tuple is relevant if it *reappears* in the result set g steps into the future, a natural formalization of "the analyst will want to see this again soon."

C.2 Evaluation: nDCG as the Quality Criterion

Let π be the ranking of R_q produced by a recommender and let k be the recommendation budget. The normalized discounted cumulative gain at rank k is

$$\mathrm{nDCG}_k(\pi, q, g) = \frac{1}{Z_k} \sum_{i=1}^{k} \frac{\mathrm{rel}(\pi(i), q, g)}{\log_2(i+1)}, \tag{6}$$

where Z_k is the ideal DCG (all relevant tuples at the top). We write $\mathrm{nDCG}_k(B, \mathcal{S})$ for the mean over all valid (q, g) pairs in session $\mathcal{S}$ for recommender B.

Overlap concentration. Define the *rule-conforming overlap fraction* of a query pair $(q, q+g)$ as

$$\rho(q, g) = \frac{|R_q \cap R_{q+g} \cap \mathcal{T}^*|}{|R_q \cap R_{q+g}|}, \tag{7}$$

where $\mathcal{T}^* = \{t \in \mathcal{T} \mid \exists r : t \models X_r \cup Y_r\}$ is the set of rule-conforming tuples. When $\rho(q, g) \approx 1$, predicting overlap is essentially the same as identifying rule tuples inside R_q.

C.3 Recommender Scoring Functions

We model each recommender as a *scoring function* $s_B : R_q \times \mathcal{H}_q \to \mathbb{R}$ that induces a ranking π_B over R_q.

Random. $s_{\text{rand}}(t) \sim \text{Uniform}(0, 1)$, independent of everything.

Frequency. Let $\text{freq}_q(A_i = v) = |\{(s, u) \mid s < q,\ u \in R_s,\ u(A_i) = v\}|$ be the historical count of value v in attribute A_i across past results.

$$s_{\text{freq}}(t) = \frac{1}{d} \sum_{i=1}^{d} \text{freq}_q\big(A_i = t(A_i)\big). \tag{8}$$

Clustering. One-hot encode each tuple in R_q to a vector $\phi(t) \in \{0, 1\}^D$. Run K-means on $\{\phi(t) \mid t \in R_q\}$ to obtain centroids $\{\mu_1, \ldots, \mu_K\}$. Score each tuple by proximity to its assigned centroid's *neighborhood density*:

$$s_{\text{clust}}(t) = -\|\phi(t) - \mu_{k(t)}\|^2, \tag{9}$$

where $k(t) = \arg\min_k \|\phi(t) - \mu_k\|$. Tuples near a centroid score high; outliers score low.

Similarity. Let $\bar{\phi}_q = \frac{1}{|R_q|} \sum_{t \in R_q} \phi(t)$ be the mean encoding of the current result. Rank by cosine similarity to this centroid:

$$s_{\text{sim}}(t) = \frac{\phi(t) \cdot \bar{\phi}_q}{\|\phi(t)\|\, \|\bar{\phi}_q\|}. \tag{10}$$

Sampling. A stratified random sample of k tuples from $R_q \cup \mathcal{H}_q^{\text{flat}}$ (the union of current and past results), with each tuple weighted by its historical frequency. This interpolates between Random and Frequency depending on the history length.

MDI. The MDI score accumulates three components across the history:

$$s_{\text{MDI}}(t, q) = \alpha \cdot A(t, q) + \beta \cdot D(t, q) + \gamma \cdot N(t, q), \tag{11}$$

where $\alpha + \beta + \gamma = 1$.

Association component $A(t, q)$: Let $\mathcal{R}_q$ be association rules mined from the history up to step q. For rule $r \in \mathcal{R}_q$, let $w_r(q) = e^{-\lambda_r (q - q_r)}$ be an exponential decay weight (λ_r is the decay rate, q_r the query at which r was mined). The composite rule score is

$$I_{\text{assoc}}(r) = \lambda_1 \,\text{conf}(r) + \lambda_2 \,\text{supp}(r) + \lambda_3 \,\text{lift}(r) + \lambda_4 \, J(r), \tag{12}$$

and the association component for tuple t is

$$A(t,q) = \sum_{r \in \mathcal{R}_q} w_r(q) \, \frac{I_{\text{assoc}}(r)}{|X_r \cup Y_r|} \, \mathbf{1}[t \models X_r \cup Y_r]. \tag{13}$$

Diversity component $D(t,q)$: Measures how much adding t to a candidate summary would increase the entropy of attribute-value distributions in $\mathcal{H}_q$ (combining Shannon, Simpson, Gini, Berger–Parker, and McIntosh indices with weights $\mu_1, \ldots, \mu_5$).

Novelty component $N(t,q)$: Penalizes tuples whose attribute values are over-represented in $\mathcal{H}_q$, favoring attribute-value combinations that have appeared sparsely in the session history.

C.4 Four Workload Parameters

The behavior of each recommender is controlled by four workload parameters that can be tuned independently in a synthetic benchmark:

ρ — **signal fraction:** The fraction of tuples in typical large query results that are rule-conforming: $\rho = \mathbb{E}[|R_q \cap \mathcal{T}^*|/|R_q|]$. In our benchmark, $\rho \approx 0.20$–0.30 for large queries.

ξ — **frequency skew:** The ratio of the empirical count of common values to the count of rare (rule-antecedent) values in $\mathcal{H}_q$: $\xi = \text{freq}_q(\text{common})/\text{freq}_q(\text{rare})$. Controlled by the bias probability p_{common} (set to 0.80).

σ^2_{noise} — **noise cluster variance:** The mean intra-cluster variance of noise tuples in encoded feature space. When p_{common} is high, the noise distribution concentrates on 2–3 values per attribute, making σ^2_{noise} small.

Q^* — **required memory depth:** The minimum session length (number of queries before the evaluation gap) needed for MDI to mine association rules with sufficient frequency to produce reliable confidence estimates. Depends on $\text{supp}(r)$ and the minimum support threshold θ_s.

C.5 Failure Conditions for Baseline Recommenders

We now state conditions under which each baseline fails to identify rule-conforming overlap tuples, giving MDI a ranking advantage.

Proposition 1 (Frequency failure)**.** *Let $f_c = \text{freq}_q(\text{common})$ and $f_r = \text{freq}_q(\text{rare})$ be the mean historical counts of common and rare attribute values respectively. Fix a query result R_q containing noise tuples R_q^{noise} biased toward common values and rule tuples $R_q^* \subseteq \mathcal{T}^*$. If $\xi = f_c/f_r > 1$, then for every rule tuple $t^* \in R_q^*$ and typical noise tuple $t^n \in R_q^{\text{noise}}$,*

$$s_{\text{freq}}(t^n) > s_{\text{freq}}(t^*).$$

Proof sketch. By construction, each component $\text{freq}_q(A_i = t^n(A_i)) = f_c$ (biased toward common values) while $\text{freq}_q(A_i = t^*(A_i)) = f_r$ for the rule-specific attributes. The inequality follows directly from Eq. (8) and $\xi > 1$.

Proposition 2 (Clustering failure)**.** *Suppose the noise distribution concentrates on c values per attribute ($c \ll |\text{dom}(A_i)|$) so that $\sigma^2_{\text{noise}} \leq \delta$ for a small $\delta > 0$, and let rule tuples scatter uniformly across $|\text{dom}(A_i)|$ values. Then for any K-means solution with K clusters, at least $K - O(\sqrt{\delta})$ centroids coincide with noise cluster means, and every rule tuple t^* receives an assignment distance $\|\phi(t^*) - \mu_{k(t^*)}\|^2 \geq \Omega(1/c)$, while $\|\phi(t^n) - \mu_{k(t^n)}\|^2 \leq \delta$ for noise tuples. By Eq. (9), $s_{\text{clust}}(t^n) > s_{\text{clust}}(t^*)$.*

Proposition 3 (Similarity failure)**.** *When $|R_q^{\text{noise}}| \gg |R_q^*|$, the result centroid $\bar{\phi}_q$ is dominated by noise encodings. Since rule tuples use attribute values orthogonal to common values in the one-hot representation, $\phi(t^*) \cdot \bar{\phi}_q \approx \rho \, \|\phi(t^*)\|$, whereas $\phi(t^n) \cdot \bar{\phi}_q \approx (1-\rho) \, \|\phi(t^n)\|$. For $\rho < 1/2$, the cosine score satisfies $s_{\text{sim}}(t^n) > s_{\text{sim}}(t^*)$.*

Proposition 4 (Random failure)**.** *An unbiased random ranking of $n = |R_q|$ tuples, of which $m = |R_q^*|$ are relevant (ground truth), achieves expected nDCG*

$$\mathbb{E}[\text{nDCG}_k] \leq \frac{m/n \cdot H_k}{Z_k},$$

where $H_k = \sum_{i=1}^{k} 1/\log_2(i+1)$ is the un-normalized ideal DCG. This equals the ideal nDCG only when $m/n = 1$; for $\rho = m/n \ll 1$ (sparse ground truth), random performance degrades proportionally to ρ.

Remark on Sampling. Sampling interpolates between Random and Frequency: with sufficient history it converges to the Frequency scorer. Therefore Propositions 1 and 4 bound its performance from above and below respectively, and it fails for the same structural reason as Frequency once $\xi > 1$.

C.6 MDI's Temporal Accumulation Advantage

The key distinction of MDI is that its association component $A(t, q)$ (Eq. 13) *grows* with session history while the baselines remain stationary or degrade.

Proposition 5 (Temporal accumulation)**.** *Let $r^* = (X^* \Rightarrow Y^*)$ be a ground-truth embedded rule with confidence $c^* = \text{conf}(r^*)$ and support $s^* = \text{supp}(r^*)$. Suppose each large query in the session contains a* persistent core *of m_c rule-conforming tuples drawn i.i.d. from $\mathcal{T}^*$, and MDI mines r^* from accumulated data at step q with estimated confidence $\hat{c}_q(r^*) \to c^*$ as $q \to \infty$. Then the association component for any tuple $t^* \models X^* \cup Y^*$ satisfies*

$$A(t^*, q) \xrightarrow{q \to \infty} \frac{c^* \left(\lambda_1 + \lambda_3 \cdot c^*/s_Y^* + \lambda_4 \cdot J^*\right)}{|X^* \cup Y^*|},$$

where $s_Y^* = \text{supp}(Y^*)$ and J^* is the J-measure of r^*. Moreover, since the rule decay weight $w_{r^*}(q) = e^{-\lambda_r (q - q_{r^*})}$ is refreshed every time r^* is re-mined (that is, at every large query), the effective decay does not erode the signal over the session.

Corollary 1 (Gap advantage)**.** *Because $A(t^*, q)$ accumulates across queries regardless of the prediction gap g, while $s_{\text{freq}}(t)$, $s_{\text{clust}}(t)$, and $s_{\text{sim}}(t)$ are memoryless (computed from R_q and recent history only), MDI's advantage over these baselines is* non-decreasing *in session length q and* independent *of g. This means that at large gaps ($g = 5, 10$), MDI retains its advantage while memoryless baselines degrade because the current result R_{q+g} drifts from R_q in ways that cannot be predicted from R_q alone.*

C.7 Benchmark Construction: A Prescriptive Recipe

The preceding propositions jointly characterize when MDI wins, but the framework also prescribes how to *deliberately construct workloads that favor any given recommender*. We state this as a theorem and summarize the parameter regimes in Table 12.

Theorem 1 (Workload-determined advantage)**.** *Let* $\mathcal{B} = \{\text{MDI}, \text{Frequency}, \text{Clustering}, \text{Similarity}, \text{Random}, \text{Samplin}$ *be the recommender set. For each* $B \in \mathcal{B}$ *there exists a workload characterized by parameters* $(\rho, \xi, \sigma^2_{\text{noise}}, Q^*)$ *such that* $\text{nDCG}_k(B, \mathcal{S}) > \text{nDCG}_k(B', \mathcal{S})$ *for all* $B' \neq B$*. The dominating regime for each recommender is:*

- ***MDI:*** $\rho < 1/2$*,* $\xi \gg 1$*,* $\sigma^2_{\text{noise}} \ll 1$*,* $Q \geq Q^*(s^*, \theta_s)$*. Rule-conforming overlap tuples are rare, noise is homogeneous, and the session is long enough for rule accumulation.*
- ***Frequency:*** $\rho \approx 1$ *and* $\xi < 1$ *(rare values carry the overlap signal, but all overlap tuples have above-average frequency).*
- ***Clustering:*** *Overlap tuples form a tight, distinct cluster* ($\sigma^2_{\text{overlap}} \ll \sigma^2_{\text{noise}}$) *separated from noise in encoded space;* $K \geq 2$*.*
- ***Similarity:*** *The future result* R_{q+g} *is* more similar *to the current result centroid than any novel tuple; queries are monotonic refinements of the same analytical theme.*
- ***Random / Sampling:*** $\rho \approx 1/2$ *and the session is short* ($Q < Q^*$)*; association rules are not yet reliable, so memory provides no signal and random selection is near-optimal.*

Table 12: Workload parameter regimes that favor each recommender. ✓ = condition required; × = not required (or reversed).

Rec.	$\rho \ll 1$ sparse sig.	$\xi \gg 1$ freq. skew	$\sigma^2_{\text{n}} \ll 1$ noise hom.	$Q \geq Q^*$ deep hist.
MDI	✓	✓	✓	✓
Frequency	×	×	×	✓
Clustering	×	×	×[a]	×
Similarity	×	×	×[b]	×
Random	×	×	×	×
Sampling	×	×	×	×

[a] distinct clusters [b] centroid-aligned

Instantiation in the benchmark generator. Table 13 shows how the parameters of `generate_benchmark.py` map to the theoretical quantities. All four conditions in Theorem 1 for MDI are satisfied by construction: $\rho \approx 0.22$ for large queries; $\xi = p_{\text{common}}/(1 - p_{\text{common}}) = 4$ ($p_{\text{common}} = 0.80$); noise rows concentrate on 2 values per attribute in a 9-attribute schema so $\sigma^2_{\text{noise}} \leq 0.11$; and with $Q = 30$ queries per session (each large query contributing about 150 tuples) the expected antecedent support count after 10 large queries is $10 \times 150 \times \text{supp}(r^*) \approx 90 > 1/\theta_s = 12.5$, satisfying $Q \geq Q^*$.

Implications for real-world workloads. Real EDA sessions over operational databases frequently exhibit high ξ simply because data skew is ubiquitous, but they rarely satisfy $\rho \ll 1$ unless the analyst is explicitly drilling into a rare sub-population. This means MDI's advantage is most pronounced in *drill-down* scenarios, exactly the kind of session studied in the SDSS and SIMBA benchmarks, and shrinks toward the level of simpler baselines in broad exploratory sweeps where overlap is spread uniformly across result rows. The framework therefore not only explains the benchmark results but also signals where MDI deployment will deliver the greatest practical benefit.

Table 13: Mapping from `generate_benchmark.py` parameters to theoretical quantities.

Theory param.	**Generator param.**	**Value**
ρ (signal frac.)	rule rows / large-query size	≈ 0.22
ξ (freq. skew)	$p_{\text{common}} = 0.80$ bias	4.0
σ^2_{noise}	2 common values / 9 attributes	≤ 0.11
$\text{supp}(r^*)$	12% rule rows; conf = 0.85	≈ 0.10
Q^*	min_support = 0.08, 3000 rows	≈ 3 large queries
Core overlap m_c	persistent rule tuples	20
Small-query overlap	24–30 of 40–50 tuples	about 0.60–0.70

References

[1] Rodney M Goodman and P Smyth. Rule induction using information theory. *G. Piatetsky*, 1991.